Aqui estão os detalhes do script guiados pelas **vantagens reais** que o POO trouxe:

1. Modelagem Natural e Intuitiva (O código imita o mundo real)

A Diferença: No procedural, o nosso cérebro tem de pensar em termos de "dicionários", "listas" e "funções genéricas" que recebem dados e cospem resultados. Na POO, usamos a classe `Aluno`.

A Vantagem: O código torna-se muito mais fácil de ler e arquitetar porque passa a falar o idioma do mundo real. Nós modelamos substantivos (objetos) que têm características (atributos, como `self.nome`) e ações (métodos, como `exibir_boletim()`). Quando você lê `aluno_cadastrado.exibir_boletim()`, a intenção do código é imediata e óbvia.

2. Encapsulamento e Segurança de Dados (Evitando efeitos colaterais)

A Diferença: No procedural, as variáveis com as notas ficavam soltas (ou num dicionário) passeando de função em função. Qualquer parte do código poderia acidentalmente sobrescrever ou apagar a nota de Matemática.

A Vantagem: A classe `Aluno` cria um "escudo" ao redor das informações daquela pessoa. O dicionário `self.boletim` vive trancado dentro do objeto. A classe `SistemaCadastro` (que interage com o teclado) não mexe nas notas diretamente; ela é obrigada a "pedir" ao objeto que faça isso através do método `aluno.adicionar_notas()`. Em sistemas grandes, isso evita bugs catastróficos onde um pedaço do código corrompe os dados de outro.

3. Manutenção Centralizada e Escalabilidade (Facilidade de evolução)

A Diferença: Imagine que a escola muda a regra e a média passa a ser ponderada. No procedural, você teria de caçar no meio do código onde estão as funções que recebem essas notas e garantir que não quebram o resto do programa.

A Vantagem: Na POO, se a regra de negócio mudar, você vai a um único lugar: dentro da classe `Aluno`, no método `determinar_situacao()` ou `calcular_media()`. Você altera o cálculo ali dentro, e o resto do sistema (como a interface `SistemaCadastro` ou a função que imprime o boletim) continua a funcionar perfeitamente sem precisar de ser reescrito. O impacto das mudanças é contido.

4. Reaproveitamento de Código (A classe como "Fábrica")

A Diferença: Criar 50 alunos no procedural significa gerir 50 dicionários espalhados na memória e ter o cuidado redobrado ao passá-los para as funções de cálculo para não misturar as notas do "João" com as da "Maria".

A Vantagem: Como vimos nos seus slides, a classe serve como uma fábrica de instâncias. A partir do momento em que o molde `Aluno` está pronto, criar mil alunos exige apenas repetir `Aluno("Nome", freq)`. Cada objeto instanciado é um compartimento estanque, completamente independente. O `aluno1` e o `aluno2` têm o seu próprio `self.boletim`, permitindo reaproveitar toda a lógica matemática infinitas vezes sem misturar os dados em memória.